

PYTHON

周信宏

Python 重點

- 運算指令
- 結構指令
- 函數
- 字串處理
- 資料結構(List, Tuple, Dict, Set)
- 檔案存取
- 物件導向

算術運算子

- Python語言的算術運算式和數學運算並沒有什麼不同，其說明如下表所示：

運算子	說明	運算式範例
-	負號	-7
+	正號	+7
*	乘法	7 * 2 = 14
/	除法	7 / 2 = 3.5
//	整數除法	7 // 2 = 3
%	餘數	7 % 2 = 1
+	加法	7 + 2 = 9
-	減法	7 - 2 = 5
**	指數	2 ** 3 = 8

指定運算子

指定運算子	範例	相當的運算式	說明
=	$x = y$	N/A	指定敘述
+=	$x += y$	$x = x + y$	加法
-=	$x -= y$	$x = x - y$	減法
*=	$x *= y$	$x = x * y$	乘法
/=	$x /= y$	$x = x / y$	除法
//=	$x //= y$	$x = x // y$	整數除法
%=	$x %= y$	$x = x \% y$	餘數
**=	$x **= y$	$x = x ** y$	指數

關係運算子

- 關係運算子的範例，如下表所示：

運算子	說明	運算式範例	結果
==	等於	3 == 4	0
!=	不等於	3 != 4	1
<	小於	3 < 4	1
>	大於	3 > 4	0
<=	小於等於	3 <= 4	1
>=	大於等於	3 >= 4	0

邏輯運算子

- 邏輯運算子（**Logical Operators**）可以連接多個關係運算式來建立複雜的條件運算式，如下表所示：

運算子	範例	說明
not	not op	NOT運算，傳回運算元相反的值，True成False；False成True
and	op1 and op2	AND運算，連接的2個運算元都為True，運算式為True
or	op1 or op2	OR運算，連接的2個運算元，任一個為True，運算式為True

if單選條件敘述

- Python語言if單選條件敘述的語法，如下所示：

if 條件運算式:

 程式敘述1

 程式敘述2

...

- 上述if條件的條件運算式如果為**True**，就執行程式區塊的程式碼（縮排4個空白字元的多個程式敘述），如果為**False**，就不執行程式區塊，請注意！Python條件敘述的條件運算式外並沒有括號「()」，不要寫成：

if (條件運算式): # 錯誤寫法

 程式敘述1

- EX.

if a > b:

 print(a)

if/else二選一條件敘述

- Python語言if/else二選一條件敘述的語法，如下所示：

if 條件運算式:

 程式敘述1

...

else:

 程式敘述2

...

- 如果if的條件運算式為**True**，就執行程式敘述1和之後的程式碼；**False**就執行程式敘述2。
- EX.
 if a > b:
 print(a)
 else:
 print(b)

單行if/else條件敘述

- Python語言並沒有提供C/C++、Java和C#語言的「條件運算式」（Conditional Expressions），不過，我們可以使用單行if/else條件敘述來代替，其語法如下所示：

變數 = 變數1 if 條件運算式 else 變數2

- 上述指定敘述的「=」號右邊是單行if/else條件敘述，如果條件成立，就將變數指定成變數1的值；否則就是指定成變數2的值。
- EX.

max = a if a > b else b

多選一條件敘述

- Python語言的if/elif/else多選一條件敘述，如下所示：

```
if a < 13:  
    print("兒童")  
elif a < 20:  
    print("青少年")  
else:  
    print("成年人")
```

- 上述if/elif/else多選一條件敘述從上而下如同階梯一般，一次判斷一個if條件，如果為True，就執行程式區塊，並且結束整個多選一條件敘述；如果為False，就進行下一次判斷。

巢狀條件敘述

- 在if/else條件敘述中可以擁有其他if/else條件敘述，稱為「巢狀條件敘述」。例如：我們可以使用巢狀條件敘述判斷3個變數中，哪一個變數值最大，如下所示：

```
if a > b and a > c:  
    print("變數 a 最大!")  
else:  
    if b > c:  
        print("變數 b 最大!")  
    else:  
        print("變數 c 最大!")
```

for迴圈

- Python語言for計數迴圈的語法需要使用range()函數來計數，如下所示：

for 計數器變數 in range(超始值, 終止值 + 1):

程式敘述

- 上述for迴圈的計數器變數是for關鍵字之後的變數，迴圈的執行次數是從range()括號的起始值開始，執行到終止值為止，因為並不含終止值，所以第2個參數值是【終止值 + 1】。請注意！在range()函數的右括號後需加上「:」冒號。
- EX.

```
for i in range(0,11):  
    sum = sum + i
```

while迴圈

- Python語言while迴圈的語法，如下所示：

while 條件運算式:

程式敘述1~N

...

- 上述while迴圈是在程式區塊開頭檢查條件，如果條件為**True**就進入迴圈執行；**False**結束執行，所以迴圈執行次數是直到條件**False**為止（請別忘了條件運算式後需加上「:」冒號）。
- EX.

```
for i <= 10:  
    sum = sum + i  
    i += 1
```

建立函數

- Python語言的使用者自訂函數，是由函數標頭和程式區塊組成，其語法如下所示：

```
def 函數名稱( 參數列 ):
    程式敘述1~n
```

```
.....
```

```
    return 傳回值
```

- 上述函數使用def關鍵字定義第1列的函數標頭（Function Header），函數標頭是使用「:」冒號結束，在之後是函數的程式區塊（Function Block）。
- EX.

```
def convert_to_f(c):
    f = (9.0 * c) / 5.0 + 32.0
    return f

f = convert_to_f(30)
```

匿名函數

- Python語言的「匿名函數」(Anonymous Functions)是指函數有定義，但是沒有命名，在Python是使用lambda關鍵字定義匿名函數，所以Python的匿名函數也稱為Lambda函數。
- Python匿名函數是使用lambda關鍵字來定義，然後將它指定給變數，其基本語法如下所示：

lambda 參數列: 運算式

- 上述lambda關鍵字之後，「:」冒號之前是參數列，如果不只一個參數，請使用「,」逗號分隔，在「:」冒號之後是運算式。
- EX.

```
square = lambda x: x * x
```

```
r = square(10)
```

字串運算子 —

使用關係運算子進行字串比較

- 如同整數和浮點數，字串也可以使用關係運算子進行2個字串的比較，如下所示：

"green" == "glow"

"green" != "glow"

"green" > "glow"

"green" >= "glow"

"green" < "glow"

"green" <= "glow"

切割字串

- Python語言索引運算子還是「切割運算子」（Slicing Operator），可以從原始字串切割出所需的子字串，其語法如下所示：

str1[start:end]

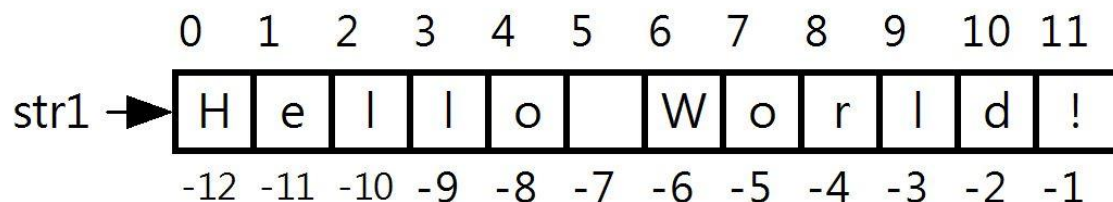
- 上述[]語法中使用「:」冒號分隔2個索引位置，可以取回字串str1從索引位置start開始到end-1之間的子字串，如果沒有start，就是從0開始；沒有end就是到字串的最後1個字元。例如：本節範例字串str1的字串內容，如下所示：

str1 = 'Hello World!'

- EX.

str1[1:3]

str1[6:-1]



字元與字串函數 - 字元函數

- Python內建字元函數主要是用來處理**ASCII**碼，其說明如下表所示：

字元函數	說明
ord()	傳回字元的ASCII碼
chr()	傳回參數ASCII碼的字元

字元與字串函數 - 字串函數

- **Python**內建字串函數可以取得字串長度、字串中的最大和最小字元，其說明如下表所示：

字串函數	說明
len()	傳回參數字串的長度
max()	傳回參數字串的最大字元
min()	傳回參數字串的最小字元

字串方法－檢查字串內容的方法

- 字串物件提供檢查字串內容的相關方法，其說明如下表所示：

字串方法	說明
isalnum()	如果字串內容是英文字母或數字，傳回True；否則為False
isalpha()	如果字串內容只有英文字母，傳回True；否則為False
isdigit()	如果字串內容只有數字，傳回True；否則為False
isidentifier()	如果字串內容是合法的識別字，傳回True；否則為False
islower()	如果字串內容是小寫英文字母，傳回True；否則為False
isupper()	如果字串內容是大寫英文字母，傳回True；否則為False
isspace()	如果字串內容是空白字元，傳回True；否則為False

字串方法－搜尋子字串方法

- 字串物件關於搜尋子字串的方法，其說明如下表所示：

字串方法	說明
endswith(str1)	如果字串內容是以參數字串str1結尾，傳回True；否則為False
startswith(str1)	如果字串內容是以參數字串str1開頭，傳回True；否則為False
count(str1)	傳回字串內容出現多少次參數字串str1的整數值
find(str1)	傳回字串內容出現參數字串str1的最小索引位置值，沒有找到傳回-1
rfind(str1)	傳回字串內容出現參數字串str1的最大索引位置值，沒有找到傳回-1

字串方法－轉換字串內容的方法

- 字串物件的轉換字串內容方法可以輸出英文大小寫轉換的字串，或是取代字串內容，其說明如下表所示：

字串方法	說明
capitalize()	傳回只有第1個英文字母大寫的字串
lower()	傳回小寫英文字母的字串
upper()	傳回大寫英文字母的字串
title()	傳回字串中每1個英文字的第1個英文字母大寫的字串
swapcase()	傳回英文字母大寫變小寫；小寫變大寫的字串
replace(old, new)	將字串中參數old的舊子字串取代成參數new的新字串

Escape 逸出字元

- 如果在Python字串內容需要輸出換行字元或一些鍵盤無法輸入的特殊字元，我們可以使用**Escape**逸出字元（**Escape Sequence**）來輸入特殊字元，這是一些使用「\」符號開頭的字串，如右表所示：

Escape 逸出字元	說明
\b	Backspace，Backspace鍵
\f	FF，Form feed換頁字元
\n	LF（Line Feed）換行或NL（New Line）新行字元
\r	Carriage Return，Enter鍵
\t	Tab鍵，定位字元
\'	「'」單引號
\"	「"」雙引號
\\	「\」符號
\?	「?」問號
\N	N是八進位值的字元常數，例如： ：\040空白字元
\xN	N是十六進位值的字元常數，例如： ：\x20空白字元

使用format()方法格式化字串

- 字串物件的format()方法是使用範本字串（**Template**）來建立輸出字串內容，在範本字串使用「{ }」符號建立格式碼（**Format Codes**），代表替換位置，然後使用format()方法的引數來替換這些格式碼顯示的內容。

使用format()方法格式化字串

- 我們可以使用位置引數（**Position Arguments**）或關鍵字引數（**Keywords Arguments**）來替換這些格式碼顯示的內容，如下所示：

```
x, y, z = 10, 20, 30
```

```
s = "Y= {1} X={0} Z={2}".format(x, y, z)
```

```
print(s)
```

```
s = "Y= {a} X={b} Z={c}".format(b=x, a=y, c=z)
```

```
print(s)
```

- 上述程式碼因為字串字面值也是物件，所以一樣可以呼叫**format()**方法，**{0}**、**{1}**和**{2}**格式碼分別對應3個引數的位置，從**0**開始，同理，我們也可以指明引數名稱**a**、**b**和**c**，其輸出的字串，如下所示：

```
Y= 20 X=10 Z=30
```

```
Y= 20 X=10 Z=30
```

正則運算式

- 「正則運算式」（**Regular Expression**）是一個字串模版，可以用來進行字串比對，在正則運算式的模版字串中，每一個字元都擁有特殊意義，這是一種小型的字串比對語言。
- 正則運算式直譯器或稱為引擎能夠將定義的正則運算式模版字串和字串變數進行比較，引擎傳回布林值，**True**表示字串符合模版字串的定義；**False**表示不符合。

正則運算式-字元集

字元集	說明
[abc]	包含英文字母a、b或c
[abc{]	包含英文字母a、b、c或符號{
[a-z]	任何英文的小寫字母
[A-Z]	任何英文的大寫字母
[0-9]	數字0~9
[a-zA-Z]	任何大小寫的英文字母
[^abc]	除了a、b和c以外的任何字元，[^....]表示之外
\w	任何字元，包含英文字母、數字和底線，即[A-Za-z0-9_]
\W	任何不是\w的字元，即[^A-Za-z0-9_]
\d	任何數字的字元，即[0-9]
\D	任何不是數字的字元，即[^0-9]
\s	空白字元，包含不會顯示的逸出字元，例如：\n和\t等，即[\t\r\n\f]
\S	不是空白字元的字元，即[^ \t\r\n\f]

正則運算式-Escape逸出字串的特殊字元

Escape逸出字串	說明
\n	新行符號
\r	Carriage Return的Enter鍵
\t	Tab鍵
\\.、\\?、\\、\\、\\[、\\]、\\{、\\}、\\(、\\)、\\+、*、\\	在範本字串代表.、?、/、\\、[、]、{、}、(、)、+、*和 特殊功能的字元
\\xHex	十六進位的ASCII碼
\\xOct	八進位的ASCII碼

正則運算式-比較字元

比較字元	說明
^	比對字串的開始，即從第1個字元開始比對
\$	比對字串的結束，即字串最後需符合範本字串
.	代表任何一個字元
	或，可以是前後2個字元的任一個
?	0或1次
*	0或很多次
+	1或很多次
{n}	出現n次
{n,m}	出現n到m次
{n,}	至少出現n次
[...]	符合方括號中的任一個字元
[^...]	符合不在方括號中的任一個字元

正則運算式

範本字串	說明
^The	字串需要是The字串開頭，例如：These
book\$	字串需要是book字串結尾，例如：a book
note	字串中擁有note子字串
a?bc	擁有0或1個a，之後是bc，例如：abc、bc字串
a*bc	擁有0到多個a，例如：bc、abc、aabc、aaabc字串
a(bc)*	在a之後有0到多個bc字串，例如：abc、abcbc、abcbcbcb字串
(a b)*c	擁有0到多個a或b，之後是c，例如：bc、abc、aabc、aaabc字串
a+bc	擁有1到多個a，之後是bc，例如：abc、aabc、aaabc字串等
ab{3}c	擁有3個b，例如：abbbbc字串，不可以是abbc或abc
ab{2,}c	至少擁有2個b，例如：abbc、abbbbc、abbbbcb等字串
ab{1,3}c	擁有1到3個b，例如：abc、abbc和abbbbc字串
[a-zA-Z]{1,}	至少1個英文字元的字串
[0-9]{1,}、[\d]{1,}	至少1個數字字元的字串

re模組的search()方法

- Python的re模組提供正規運算式的相關方法，可以使用正規運算式的範本字串來比對字串內容。首先需要匯入re模組，如下所示：

```
import re
```

re模組的search()方法-使用search()方法

- 在re模組的re.search()方法可以在字串中找到範本字串中第1個符合的子字串，如下所示：

```
match = re.search(r"[\w.-]+@[A-Za-z0-9_.-]+",  
str1)
```

- 上述search()方法的第2個參數是目標字串，第1個參數是範本字串，因為內含「\」符號字元，我們需要使用「原料字串」（Raw Strings），也就是不會作任何處理的原始字串，如下所示：

```
r"[\w.-]+@[A-Za-z0-9_.-]+"
```

- 如果找到符合的子字串，我們可以呼叫match.group()方法顯示找到的子字串，如下所示：

```
print(match.group())
```

re模組的search()方法- 群組抓取 (Group Capturing)

- 群組抓取可以取出正規運算式找到符合字串的部分內容，我們需要在範本字串使用「()」括號預先定義需要取出的部分內容，如下所示：

```
r"([\w.-]+)@([A-Za-z0-9_.-]+)"
```

- 上述程式碼的「@」符號前後都使用「()」括號括起，共有2個部分，所以，呼叫group()方法傳入引數值1，就是第1部分，2是第2部分，如下所示：

```
print(match.group())
```

```
print(match.group(1))
```

```
print(match.group(2))
```

re模組的findall()方法

- 在re模組的search()方法只能找到第1個符合的字串，如果需要找出所有符合的子字串，請使用findall()方法，如下所示：

```
match = re.findall(r"[\w.-]+@[A-Za-z0-9_.-]+", str1)
```

- 上述findall()方法的參數和search()方法相同，可以傳回所有符合子字串的清單。

建立清單

- Python語言的「清單」（**Lists**）類似其他程式語言「陣列」（**Arrays**），這是一種循序資料結構，中文譯名有清單、串列和列表等。
- 不同於字串型態的不能更改，清單允許更改（**Mutable**）內容，我們可以新增、刪除、插入和更改清單的項目（**Items**）。
- Python清單（**Lists**）是使用「`[]`」方括號括起的多個項目，每一個項目使用「`,`」逗號分隔。

建立清單

- 我們可以使用指定敘述指定變數值是清單，清單項目可以是相同資料型態，也可以是不同資料型態，如下所示：

```
list1 = []
```

```
list2 = [1, 2, 3, 4, 5]
```

```
list3 = [1, 'Hello', 3.5]
```

- 上述第1列程式碼是空清單，第2個清單項目都是整數，第3個清單的項目是不同資料型態。我們也可以使用物件方式來建立清單，如下所示：

```
list4 = list()
```

```
list5 = list(["tom", "mary", "joe"]) (clone)
```

```
list6 = list("python") (to char list)
```

清單包含

- 清單包含（**List Comprehension**）是一種簡潔語法來建立清單，我們可以在「`[]`」方括號中使用**for**迴圈產生清單項目，如果需要，還可以加上**if**條件子句篩選出所需的項目，如下所示：

```
list1 = [x for x in range(10)]
```

- 上述程式碼的第**1**個變數**x**是清單項目，這是使用之後**for**迴圈來產生項目，以此例是**0~9**，換句話說，我們可以建立清單：`[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`。

建立清單－輸出清單項目

- Python可以使用print()函數輸出清單項目，如下所示：

```
print(list2)
```

```
print("list3 = " + str(list3))
```

- 上述print()函數可以直接輸出清單變數的內容（print()函數會自動轉換成字串），我們也可以呼叫str()函數建立字串連接運算式來輸出清單項目。

存取與走訪清單項目 – 走訪清單的每一個項目

- 我們可以使用**for**迴圈走訪顯示清單的每一個項目，如下所示：

```
for e in list1:  
    print(e, end=" ")
```

- 上述**for**迴圈可以一一取出清單每一個項目和顯示出來。

存取與走訪清單項目 – 存取和走訪巢狀清單

- 如果是多層的巢狀清單，我們需要使用多個索引值來存取指定項目，如下所示：

```
list2 = [[2, 4], ["tom", "mary", "joe"], [1, 3, 5]]
```

- 上述巢狀清單有2層，第1層有3個項目，每一個項目是另一個清單，所以存取指定項目需要使用2個索引，例如：取得第2個項目中的第1個項目，和更改第3個項目中的第2個項目，如下所示：

```
list2[1][0]
```

```
list2[2][1] = 7
```

- 因為巢狀清單有兩層，使用2層for迴圈走訪每一個項目，如下所示：

```
for e1 in list2:
```

```
    for e2 in e1:
```

```
        print(e2, end=" ")
```

插入、新增與刪除清單項目 — 在清單新增項目

- 我們可以呼叫清單物件的 **append()** 方法新增單一項目，如下所示：

```
list1.append(7)
```

- 上述方法新增項目 7，現在的清單是：**[1, 5, 7]**。如果需要同時新增多個項目，請使用 **extend()** 方法，如下所示：

```
list1.extend([9, 11, 13])
```

- 上述方法擴充清單的項目，一次就新增 3 個項目，現在的清單是：**[1, 5, 7, 9, 11, 13]**。

插入、新增與刪除清單項目 – 在清單插入項目

- 在清單新增項目是新增在清單的最後，我們也可以使用 `insert()` 方法在指定的索引位置插入1個項目，如下所示：

`list1.insert(1, 3)`

- 上述方法是在第1個參數的索引值插入第2個參數的項目，即插入第2個項目3，現在的清單是：`[1, 3, 5, 7, 9, 11, 13]`。

插入、新增與刪除清單項目 – 刪除清單項目1

- 我們可以使用**del**關鍵字，如下所示：刪除指定索引的清單項目

```
del list1[2]
```

- 上述程式碼刪除第3個項目，現在的清單是：**[1, 3, 7, 9, 11, 13]**。除了**del**關鍵字，我們也可以使用**pop()**方法刪除和傳回最後1個項目，如下所示：

```
e1 = list1.pop()
```

- 上述方法**刪除最後1個項目和傳回**，所以變數**e1**就是最後1個項目**13**，現在的清單是：**[1, 3, 7, 9, 11]**。

插入、新增與刪除清單項目 – 刪除清單項目2

- 如果`pop()`方法有參數，就是刪除和傳回指定索引值的項目，如下所示：

```
e2 = list1.pop(1)
```

- 上述方法刪除索引值`1`的第`2`個項目和傳回，所以變數`e2`就是第`2`個項目`3`，現在的清單是：`[1, 7, 9, 11]`。如果需要刪除指定項目（不是索引），我們可以使用`remove()`方法，如下所示：

```
list1.remove(9)
```

- 上述方法刪除項目`9`，現在的清單是：`[1, 7, 11]`。

清單運算子 - 連接運算子

- 當算術運算子的「+」加法使用在清單，就是**連接2個清單**，即**合併清單**，如下所示：

```
list1 = [2, 4]
```

```
list2 = [6, 8, 10]
```

```
list3 = list1 + list2
```

- 上述程式碼使用清單連接運算子連接清單list1和list2，清單list3的項目是：[2, 4, 6, 8, 10]。

清單運算子 - 重複運算子

- 當算術運算子的「*」乘法使用在清單，可以建立重複第2個運算元次數的清單，如下所示：

`list4 = list1 * 3`

- 上述程式碼可以重複3次list1的項目，清單list4的項目是：
`[2, 4, 2, 4, 2, 4]`。

清單運算子 - 成員運算子

- 成員運算子 **in** 和 **not in** 也可以使用在清單，用來檢查清單是否存在指定的項目，如下所示：

```
print(8 in list2)    # True
```

```
print(2 not in list1) # False
```

- 上述程式碼可以檢查項目 **8** 是否存在 **list2** 清單，項目 **2** 是否存在 **list1** 清單。

切割清單－切割運算子

- 切割運算子可以從原始清單切割出所需的子清單，其語法和第8-1-3節相同，筆者就不重複說明。本節範例清單的項目是字串內容'Hello World!'的每一個字元，如下所示：

```
list1 = list('Hello World!')
```

- 上述程式碼建立的清單項目為：['H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd', '!']，項目的索引位置值可以是正，也可以是負值，如下圖所示：

	0	1	2	3	4	5	6	7	8	9	10	11
list1 →	['H', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd', '!']											
	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

切割清單－切割清單的範例

- 現在，就讓我們看一些切割清單`list1 = list('Hello World!')`的範例，如下表所示：

切割清單	索引值範圍	取出的子清單
<code>list1[1:3]</code>	1~2	<code>['e', 'l']</code>
<code>list1[1:5]</code>	1~4	<code>['e', 'l', 'l', 'o']</code>
<code>list1[:7]</code>	0~6	<code>['H', 'e', 'l', 'l', 'o', ' ', 'W']</code>
<code>list1[4:]</code>	4~11	<code>['o', ' ', 'W', 'o', 'r', 'l', 'd', '!']</code>
<code>list1[1:-1]</code>	1~(-2)	<code>['e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd']</code>
<code>list1[6:-2]</code>	6~(-3)	<code>['W', 'o', 'r', 'l']</code>

元組

- Python的「元組」(Tuple)是一種類似清單的容器型態，簡單的說，元組是一個**唯讀的清單**，一旦指定元組的項目，就不再允許更改元組的項目。在實務上，Python使用元組對比清單的優點，如下所示：
 - 因為元組的項目不允許更改，走訪元組比走訪清單更有效率，可以輕微的增加程式的執行效能。
 - 元組因為**項目不允許更改**，**可以作為字典的鍵 (Keys)** 來使用，但是**清單不可以**。
 - 如果程式需要使用不允許更改的唯讀清單，我們可以使用元組來實作，而且保證項目不會被更改。

建立元組

- **Python**的元組（**Tuple**）是使用「**()**」括號來建立，每一個項目使用「**,**」逗號分隔。我們可以使用指定敘述指定變數值是一個元組，元組的項目可以是相同資料型態，也可以是不同資料型態，如下所示：

```
t1 = ()
```

```
t2 = (1, 2, 3, 4, 5)
```

```
t3 = (1, 'Hello', 3.5)
```

- 上述第**1**列程式碼是空元組，第**2**個元組項目都是整數，第**3**個元組的項目是不同的資料型態。我們也可以使用物件方式來建立元組，如下所示：

```
t4 = tuple()
```

```
t5 = tuple(["tom", "mary", "joe"])
```

```
t6 = tuple("python")
```

建立元組 -輸出元組項目

- Python可以使用print()函數輸出元組項目，如下所示：

```
print(t2)
```

```
print("t3 = " + str(t3))
```

- 上述print()函數可以直接輸出元組變數的內容，我們也可以呼叫str()函數轉換成字串型態來輸出元組項目。

取出與走訪元組項目 – 使用索引運算子取出元組項目

- **Python**元組可以使用「`[]`」索引運算子取出指定位置的項目，索引值是從**0**開始，可以是負值，如下所示：

```
t1 = (1, 2, 3, 4, 5, 6)
```

```
print(t1[0]) # 1
```

```
print(t1[1]) # 2
```

```
print(t1[-1]) # 6
```

```
print(t1[-2]) # 5
```

- 上述程式碼依序顯示元組**t1**的第**1**和第**2**個項目，**-1**是最後**1**個，**-2**是倒數第**2**個。

取出與走訪元組項目 – 走訪元組的每一個項目

- 我們可以使用**for**迴圈走訪顯示元組的每一個項目，如下所示：

```
for e in t1:
```

```
    print(e, end=" ")
```

- 上述**for**迴圈可以一一取出元組每一個項目和顯示出來。

元組運算子 - 連接運算子

- 當算術運算子的「+」加法使用在元組，就是連接2個元組，即合併元組，如下所示：

t1 = (2, 4)

t2 = (6, 8, 10)

t3 = t1 + t2

- 上述程式碼使用清單連接運算子連接清單t1和t2，清單t3的項目是：(2, 4, 6, 8, 10)。

元組運算子 - 重複運算子

- 當算術運算子的「*」乘法使用在元組，可以建立重複第2個運算元次數的元組，如下所示：

t4 = t1 * 3

- 上述程式碼可以重複3次t1的項目，清單t4的項目是：(2, 4, 2, 4, 2, 4)。

元組運算子 - 成員運算子

- 成員運算子`in`和`not in`也可以使用在元組，用來檢查元組是否存在指定的項目，如下所示：

```
print(8 in t2)      # True
```

```
print(2 not in t1)  # False
```

- 上述程式碼可以檢查項目`8`是否存在`t2`元組，項目`2`是否不存在`t1`元組。

切割元組

- 如同清單和字串，元組一樣可以使用切割運算子「**[start:end]**」從原始元組切割出所需的子元組。本節範例元組的項目是字串內容'**Hello World!**'的每一個字元，如下所示：

t1 = tuple('Hello World!')

- 上述程式碼建立的元組項目為：**('H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd', '!')**，項目的索引位置值可以是正，也可以是負值，如下圖所示：

	0	1	2	3	4	5	6	7	8	9	10	11
t1 →	('H', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd', '!')											
	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

切割元組 — 切割元組的範例

- 現在，就讓我們看一些切割元組的範例，如下表所示：

切割元組	索引值範圍	取出的子元組
t1[1:3]	1~2	('e', 'l')
t1[1:5]	1~4	('e', 'l', 'l', 'o')
t1[:7]	0~6	('H', 'e', 'l', 'l', 'o', ' ', 'W')
t1[4:]	4~11	('o', ' ', 'W', 'o', 'r', 'l', 'd', '!')
t1[1:-1]	1~(-2)	('e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd')
t1[6:-2]	6~(-3)	('W', 'o', 'r', 'l')

元組函數與元組方法

元組函數	說明
len()	傳回參數元組的長度，即項目數
max()	傳回參數元組的最大項目
min()	傳回參數元組的最小項目
tuple()	傳回參數字串、清單、字典和集合轉換成的元組
enumerate()	傳回enumerate物件，其內容是元組索引和項目的元組
sum()	傳回參數元組項目的總和
sorted()	傳回排序參數元組的一個全新元組

元組函數與元組方法

- **Python**元組類別提供方法搜尋項目和計算出現次數。常用元組方法的說明，如下表所示：

元組方法	說明
<code>count(item)</code>	傳回元組中等於參數item項目的個數
<code>index(item)</code>	傳回元組第1個找到參數item項目的索引， 如果項目不存在，就會產生ValueError錯誤

字典

- Python的「字典」（**Dictionaries**）是一種儲存鍵值資料的容器型態，我們可以使用鍵（**Key**）來取出和更改值（**Value**），或使用鍵來新增和刪除項目。
- 基本上，字典和清單一樣，其儲存項目是可以更改的，對比其他程式語言，就是結合陣列（**Associative Array**）。

建立字典

- Python的字典（Dictionaries）是使用大括號「`{}`」定義成對的鍵和值（Key-value Pairs），每一對使用「`,`」逗號分隔，其中的鍵和值是使用「`:`」冒號分隔，如下所示：

```
{  
    "key1": "value1",  
    "key2": "value2",  
    "key3": "value3",  
    ...  
}
```

- 上述key1~3的值必須是唯一，資料型態只能是字串、數值和元組型態。

建立字典

- 我們可以使用指定敘述指定變數值是一個字典，字典項目的鍵和值可以是相同資料型態，也可以是不同資料型態，如下所示：

```
d1 = {}
```

```
d2 = {1: 'apple', 2: 'ball'}
```

```
d3 = {  
    "name": "tom",  
    1: [1, 3, 5]  
}
```

- 上述第1列程式碼是空字典，第2個字典項目的鍵是整數；值是字串，第3個字典的項目是不同的資料型態，鍵是字串和數值；值是字串和清單（請注意！大括號隱含有加上「\」符號，所以可以直接分成多列來編排）。

建立字典

- 我們也可以使用物件方式來建立字典，如下所示：

```
d4 = dict()
```

```
d5 = dict([(1,"tom"), (2,"mary"), (3, "joe")])
```

- 上述第**1**列程式碼是建立空字典，第**2**列是使用清單建立字典，每一個項目是一個**2**個項目的元組。

字典包含

- 字典包含（**Dictionary Comprehension**）是一種簡潔語法來建立字典，我們可以在「**{}**」大括號中使用**for**迴圈產生字典項目，如果需要，還可以加上**if**條件子句來篩選出所需的項目（**Python**程式：**Ch9_4_2.py**），如下所示：

```
d1 = {x:x*x for x in range(10)}
```

- 上述程式碼的第**1**個**x:x*x**是字典項目，位在「**:**」前是鍵；之後是值，這是使用之後**for**迴圈產生項目，以此例是**0~9**，換句話說，我們可以建立字典：**{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81}**。

建立字典－輸出字典項目

- Python可以使用`print()`函數輸出字典項目，如下所示：

```
print(d2)
```

```
print("d3 = " + str(d3))
```

- 上述`print()`函數可以直接輸出字典變數的內容，我們也可以呼叫`str()`函數轉換成字串型態來輸出字典項目。

取出、更改、新增與走訪字典項目

- Python字典可以使用「[]」索引運算子存取指定鍵的項目，如下所示：

```
d1 = {  
    "name": "tom",  
    "age": 20,  
    1: [1, 3, 5]  
}
```

```
print(d1["name"])
```

```
print(d1["age"])
```

```
print(d1[1])
```

- 上述程式碼依序顯示字典d1的鍵是"name"、"age"和1項目的值。

取出、更改、新增與走訪字典項目

- 更改字典項目的值是使用指定敘述「=」等號，例如：更改鍵"age"的值成為21，如下所示：

```
d1["age"] = 21
```

取出、更改、新增與走訪字典項目

- 如果使用指定敘述「=」等號更改的鍵不存在，就是新增字典的項目，如下所示：

```
d1["address"] = "台北"
```

- 上述程式碼新增鍵為"address"；值是"台北"的項目。

取出、更改、新增與走訪字典項目

- 我們可以使用**for**迴圈走訪顯示字典的每一個項目值，如下所示：

```
for key in d1:  
    print(d1[key], end=" ")
```

- 上述**for**迴圈可以一一取出字典的每一個鍵，然後使用鍵取出項目值。

刪除字典項目

- Python字典可以使用del關鍵字和相關方法來刪除字典項目。
本節範例字典d1原來有5個項目，如下所示：

```
d1 = {1:1, 2:4, "name":"tom", "age":20, 5:22}
```

刪除字典項目

- 我們可以使用`del`關鍵字刪除指定鍵的項目，如下所示：

```
del d1[2]
```

```
del d1["age"]
```

- 上述程式碼刪除鍵2和"age"的項目，現在的字典是：`{1:1, "name":"tom", 5:22}`。

刪除字典項目

- 我們也可以使用`pop()`方法刪除參數鍵和傳回項目值，如下所示：

```
e1 = d1.pop(5)
```

- 上述方法刪除鍵5項目和傳回值，所以變數`e1`就是項目值22，現在的字典是：`{1:1, "name":"tom"}`。

刪除字典項目

- `popitem()`方法可以刪除最後**1**個項目和傳回，如下所示：

```
e2 = d1.popitem()
```

刪除字典項目

- 我們可以使用**clear()**方法刪除字典的所有項目，也就是成為空字典，如下所示：

d1.clear()

字典運算子

- Python字典可以使用成員運算子判斷項目是否存在，關係運算子的「==」和「!=」判斷2個字典是否相等。本節範例字典d1和d2，如下所示：

```
d1 = {"tom":30, "bobe":3}
```

```
d2 = {"bobe":3, "tom":30}
```

字典運算子 — 成員運算子

- 成員運算子 **in** 和 **not in** 可以使用在字典，用來檢查字典是否存在指定的鍵，如下所示：

```
print("tom" in d1)      # True
```

```
print("tom" not in d1)  # False
```

- 上述程式碼可以檢查鍵 **"tom"** 是否存在字典 **d1**，是否不存在字典 **d1**。

字典運算子 — 關係運算子

- 兩個字典可以使用關係運算子的「==」和「!=」判斷2個字典是否相等，或不相等（字典並不支援其他的關係運算子），如下所示：

```
print(d1 == d2) # True
```

```
print(d1 != d2) # False
```

字典函數與字典方法

- **Python**語言內建字典函數可以取得字典長度的項目數、建立字典和排序字典的鍵等。常用字典函數的說明，如下表所示：

字典函數	說明
len()	傳回參數字典的長度，即項目數
dict()	傳回參數轉換成的字典
sorted()	傳回排序參數字典項目鍵的一個全新清單

字典函數與字典方法

- Python字典類別提供方法刪除項目、取出鍵、取出值和清除字典。常用字典方法的說明，如下表所示：

字典方法	說明
pop(key)	刪除和傳回字典中參數key鍵的項目值，如果key鍵不存在，就會產生KeyError錯誤
popitem()	刪除最後1個項目和傳回
clear()	刪除字典中的所有項目
get(key)	傳回字典中參數key鍵的項目值，如果key鍵不存在，傳回None，也可以指定第2個參數是當沒有key鍵時，傳回的字串
keys()	傳回字典中所有鍵的dict_keys物件
values()	傳回字典中所有值的dict_values物件

集合

- Python的「集合」（**Sets**）是一種無順序的元素集合，每一個元素是唯一；不可重複，我們可以更新、新增和刪除元素，和執行數學集合運算：交集、聯集和差集等。
- Python語言的集合（**Sets**）是使用「**{}**」大括號括起，每一個元素是使用「**,**」逗號分隔。

建立集合

- 我們可以使用指定敘述指定變數值是一個集合，集合元素可以是相同資料型態，也可以是不同的資料型態，如下所示：

```
s1 = {1, 2, 3}
```

```
s2 = {1, 'Hello', 3.5}
```

- 上述第1列程式碼的集合元素都是整數，第2個集合的元素是不同的資料型態。我們也可以使用物件方式來建立元組，如下所示：

```
s3 = set()
```

```
s4 = set(["tom", "mary", "joe"])
```

```
s5 = set("Python")
```

- 上述第1列程式碼是建立空集合（請注意！`{}`空大括號是空字典，並不是空集合），第2列是使用清單建立集合，最後是使用字串的每一個字元來建立集合。

建立集合

- Python可以使用`print()`函數輸出集合元素，如下所示：

```
print(s1)
```

```
print("s2 = " + str(s2))
```

- 上述`print()`函數可以輸出集合元素，我們也可以呼叫`str()`函數轉換成字串型態來輸出集合元素。

建立集合

- 我們可以使用**for**迴圈走訪顯示集合中的每一個元素，如下所示：

```
for e in s1:  
    print(e, end=" ")
```

新增和刪除集合元素

- Python集合可以呼叫相關方法來新增和刪除集合元素。本節範例集合**s1**原來有**2**個元素，如下所示：

```
s1 = {1, 5}
```

新增和刪除集合元素

- 我們可以呼叫集合物件的**add()**方法來新增單一元素，如下所示：

s1.add(7)

- 上述方法新增元素7，現在的集合是：**{1, 5, 7}**。如果需要同時新增多個元素，請使用**update()**方法，如下所示：

s1.update([9, 11, 13])

- 上述方法使用參數的清單來新增集合元素，一次就新增**3**個元素，現在的集合是：**{1, 5, 7, 9, 11, 13}**。

新增和刪除集合元素

- `update()` 方法的參數可以同時使用清單，和集合來新增元素，如下所示：

`s1.update([2, 3], {4, 6, 10})`

- 上述方法使用參數的清單和集合新增集合元素，一次就新增5個元素，現在的集合是：`{1, 2, 3, 4, 5, 6, 7, 9, 10, 11, 13}`。

新增和刪除集合元素

- Python集合可以使用`discard()`和`remove()`方法刪除集合元素，這2個方法的功能相同，不過，如果刪除的元素不存在，`discard()`方法不會有錯誤；`remove()`方法會產生錯誤，如下所示：

`s1.discard(2)`

`s1.remove(6)`

- 上述2列程式碼刪除元素2和6，現在的集合是：`{1, 3, 4, 5, 7, 9, 10, 11, 13}`。

新增和刪除集合元素

- 我們也可以使用`pop()`方法隨機刪除和傳回任一個元素，如下所示：

```
e1 = s1.pop()
```

- 如果需要刪除集合的全部元素，我們可以使用`clear()`方法，如下所示：

```
s1.clear()
```

集合運算

- Python語言的集合可以執行集合運算的交集、聯集和差集。
本節測試的2個集合，如下所示：

$A = \{1, 2, 3, 4, 5\}$

$B = \{4, 5, 6, 7, 8\}$

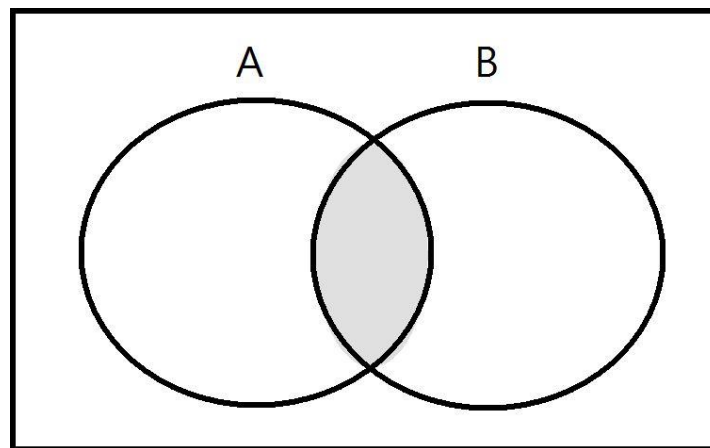
集合運算 – 交集（Set Intersection）

- 交集是2個集合都存在元素的集合，Python交集是使用「&」運算子或intersection()方法，如下所示：

$C = A \& B$

$C = A.intersection(B)$

- 上述運算式結果的集合是： $\{4, 5\}$ 。



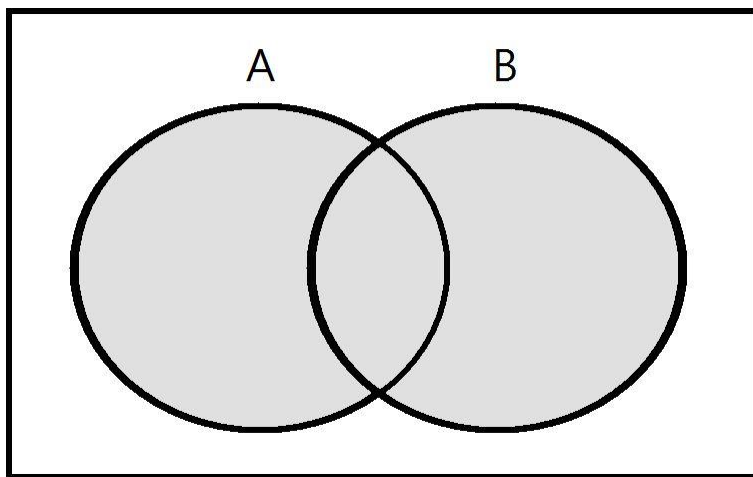
集合運算 – 聯集 (Set Union)

- 聯集是2個集合所有不重複元素的集合，Python聯集是使用「|」運算子或union()方法，如下所示：

$$C = A | B$$

$$C = A.union(B)$$

- 上述運算式結果的集合是： $\{1, 2, 3, 4, 5, 6, 7, 8\}$ 。



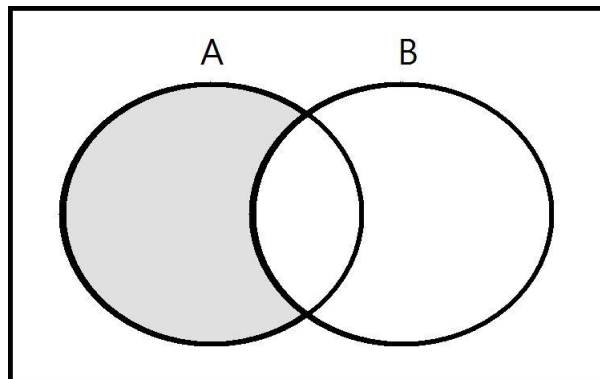
集合運算 – 差集 (Set Difference)

- 差集是2個集合 $A - B$ ，只存在集合A，不存在集合B的元素集合（ $B - A$ 就是存在B；不存在A），Python差集是使用「-」運算子或difference()方法，如下所示：

$$C = A - B$$

$$C = A.\text{difference}(B)$$

- 上述運算式結果的集合是： $\{1, 2, 3\}$ 。



集合運算 –

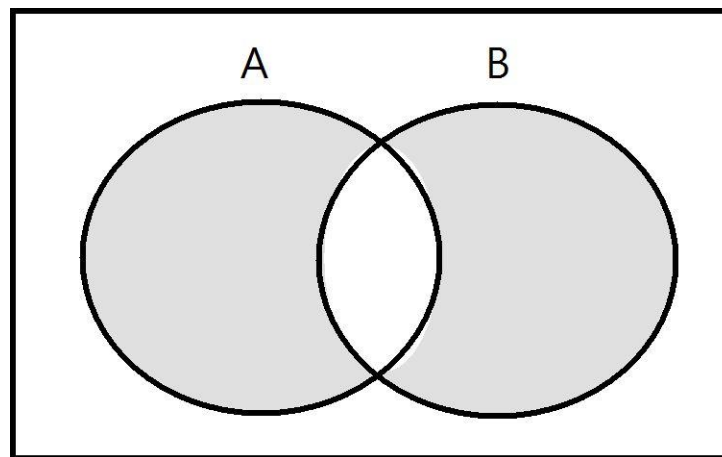
對稱差集 (Set Symmetric Difference)

- 對稱差集是2個集合的元素，不包含2個集合都擁有的元素，Python對稱差集是使用「^」運算子或 `symmetric_difference()` 方法，如下所示：

$$C = A \wedge B$$

$$C = A.\text{symmetric_difference}(B)$$

- 上述運算式結果的集合是： $\{1, 2, 3, 6, 7, 8\}$ 。



集合的成員運算子

- 成員運算子`in`和`not in`可以使用在集合，用來檢查集合是否存在指定的元素（Python程式：Ch9_3_4.py），如下所示：

```
s1 = {"tom", "bob", "joe"}
```

```
print("tom" in s1)      # True
```

```
print("tom" not in s1)  # False
```

- 上述程式碼可以檢查鍵`"tom"`是否存在集合`s1`，是否不存在集合`s1`。

集合函數與集合方法

- 集合函數和清單函數相同，筆者就不重複說明。常用方法說明，如下表所示：

集合方法	說明
<code>isdisjoint()</code>	如果2個集合沒有交集傳回True
<code>issubset()</code>	如果是另一集合的子集合傳回True
<code>issuperset()</code>	如果集合包含另一個集合傳回True

物件

- 物件導向程式是使用物件來建立程式，每一個物件儲存資料（**Data**；變數）和提供行為（**Behaviors**；函數），透過物件之間的通力合作來完成程式的功能。

定義類別

- 類別（**Class**）是物件的模子，也是藍圖，我們需要先定義類別，才能依據類別的模子來建立物件。在Python語言是使用**class**關鍵字來定義類別，如下所示：

```
class Student:
```

```
    def __init__(self, name, grade):
```

```
        self.name = name
```

```
        self.grade = grade
```

```
    def displayStudent(self):
```

```
        print("姓名 = " + self.name)
```

```
        print("成績 = " + str(self.grade))
```

```
    def whoami(self):
```

```
        return self.name
```

使用類別建立物件

- 在定義類別後，我們就可以使用類別來建立物件，也稱為實例（**Instances**），同一類別可以如同工廠生產一般的建立多個物件，如下所示：

```
s1 = Student("John", 85)
```

- 上述程式碼建立物件s1，Student()就是呼叫Student類別的建構子方法，擁有2個參數來建立物件，然後可以使用「.»運算子呼叫物件方法，如下所示：

```
s1.displayStudent()
```

```
print("s1.whoami() = " + s1.whoami())
```

- 同樣語法，我們可以存取物件的資料欄位，如下所示：

```
print("s1.name = " + s1.name)
```

```
print("s1.grade = " + str(s1.grade))
```

隱藏資料欄位

- **Python**類別定義的資料欄位和方法預設可以在其他**Python**程式碼存取這些資料欄位，和呼叫這些方法，對比其他物件導向程式語言就是**public**公開成員。
- 如果資料欄位需要隱藏，或方法只能在類別中呼叫，並不是類別對外的使用介面，我們需要使用**private**私有成員，在**Python**資料欄位和方法名稱只需使用2個「**_**」底線開頭，就表示是私有（**Private**）資料欄位和方法。

隱藏資料欄位

- 建構子的__grade資料欄位是隱藏的資料欄位），如下所示：

```
def __init__(self, name, grade):  
    self.name = name  
    self.__grade = grade
```

- 我們也可以建立只有在類別中呼叫的私有方法（**Private Methods**），如下所示：

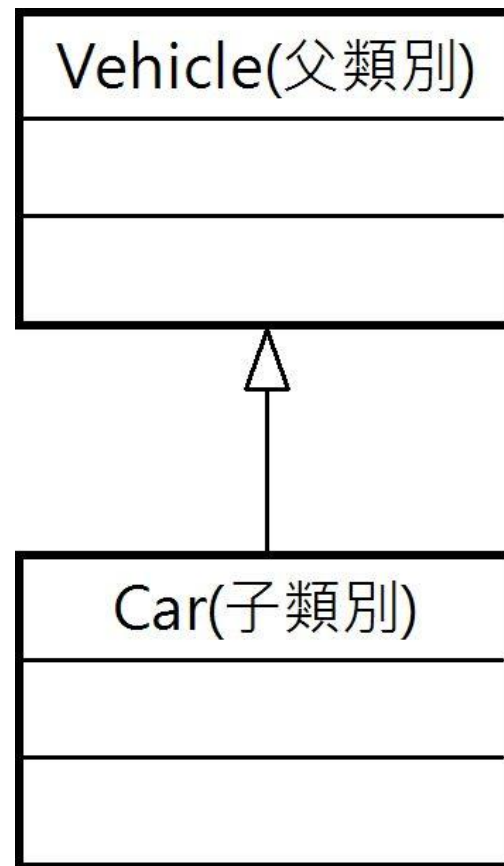
```
def __getGrade(self):  
    return self.__grade
```

- 方法名稱是__getGrade()，這個方法只能在定義類別的程式碼來呼叫，呼叫時記得一樣需要加上**self**，如下所示：

```
print("成績 = " + str(self.__getGrade()))
```

類別的繼承

- 類別如果是繼承自其他類別，我們稱此類別為繼承類別的「子類別」（**Subclass**），繼承的類別稱為「父類別」（**Superclass**）或「基礎類別」（**Base Class**），例如：類別**Car**繼承自類別**Vehicle**，其繼承關係如右圖所示：



類別的繼承 — 父類別Vehicle

```
class Vehicle:
    def __init__(self, name, color):
        self.__name = name
        self.__color = color
    def getColor(self):
        return self.__color
    def setColor(self, color):
        self.__color = color
    def getName(self):
        return self.__name
```

類別的繼承 — 子類別Car

- 在子類別**Car**定義是繼承父類別**Vehicle**，如下所示：

```
class Car(Vehicle):  
    def __init__(self, name, color, model):  
        # 呼叫父類別的建構子  
        super().__init__(name, color)  
        self.__model = model  
    def displayCar(self):  
        print("名稱 = " + self.getName())  
        print("車型 = " + self.__model)  
        print("顏色 = " + self.getColor())
```

覆寫方法

- 父類別如果有不適用的方法，在子類別可以定義同名和相同參數列的方法來取代，稱為「覆寫方法」（**Overriding Methods**）。

覆寫方法

- 例如：父類別的`displayVehicle()`方法並沒有車型資料，在子類別可以定義同名和同參數列的`displayVehicle()`方法來覆寫，如下所示：

```
class Vehicle:    # 父類別
    ...
    def displayVehicle(self):
        print("廠牌 = " + self.__name)
        print("顏色 = " + self.__color)
class Car(Vehicle): # 子類別
    ...
    def displayVehicle(self):
        print("名稱 = " + self.getName())
        print("車型 = " + self.__model)
        print("顏色 = " + self.getColor())
```

開啟檔案

- 在Python程式可以使用open()函數開啟檔案，如下所示：

```
fp = open("note.txt", "w")
```

- 上述函數的第1個參數是檔案名稱或檔案完整路徑，請注意！路徑「\」符號在Windows作業系統需要使用逸出字元「\\」（Python程式：Ch11_1_1a.py），如下所示：

```
fp = open("temp\\note.txt", "w")
```

- 上述參數"temp\\note.txt"就是路徑「temp\note.txt」，第2個參數是檔案開啟的模式字串。

關閉檔案

- 在執行完檔案操作後，請使用檔案指標**fp**的檔案物件執行**close()**方法來關閉檔案，如下所示：

fp.close()

寫入資料

- 在Python程式成功開啟檔案後，我們可以呼叫write()方法將參數字串寫入檔案，如下所示：

```
fp = open("note.txt", "w")
```

```
fp.write("john\n")
```

```
fp.write("mary\n")
```

```
fp.close()
```

- 上述程式碼在開啟寫入檔案note.txt後，呼叫2次檔案物件的write()方法來寫入資料，請注意！不同於print()函數預設加上換行的「\n」新行字元，write()方法如需換行，我們需要自行在字串後加上新行字元。

新增資料

- 如果想在檔案現有資料的最後新增資料，例如：我們準備在note.txt檔案最後再新增一名姓名資料，此時，我們需要使用"a"模式字串開啟新增檔案，如下所示：

```
fp = open("note.txt", "a")
```

```
fp.write("paul\n")
```

```
fp.close()
```

- 上述程式碼的open()函數是使用"a"模式字串，所以write()方法寫入的字串是在現有檔案的最後，也就是新增資料至檔尾。

讀取檔案的全部內容-使用read()方法

- 檔案物件的read()方法如果沒有參數，就是讀取檔案的全部內容，如下所示：

```
str1 = fp.read()  
print("檔案內容:")  
print(str1)
```

- 上述程式碼讀取整個檔案成為一個字串，然後顯示字串內容。

讀取檔案的全部內容-使用readlines()方法

- 我們也可以使用檔案物件的**readlines()**方法，讀取檔案內容成為清單，每一行是一個項目，如下所示：

```
list1 = fp.readlines()
```

```
print("檔案內容:")
```

```
print(list1)
```

```
for line in list1:
```

```
    print(line, end="")
```

- 上述程式碼讀取檔案內容的清單後，使用**for**迴圈顯示每一行的檔案內容，因為檔案中的每一行有換行，所以**print()**函數就不需要換行。

讀取檔案的部分內容

- 檔案物件的`read()`方法可以加上參數值來讀取所需的字數，如下所示：

```
str1 = fp.read(1)
```

```
str4 = fp.read(2)
```

- 上述程式碼從目前的檔案指標讀取**1**個字和**2**個字，如果是中文字佔**2**個字元，英文字母是**1**個。我們也可以使用`readline()`方法只讀取**1**行，如下所示：

```
str2 = fp.readline()
```

- 上述程式碼讀取目前檔案指標至此行最後**1**個字元（含新行字元「`\n`」）的一行內容，每呼叫**1**次可以讀取**1**行。

開啟和關閉二進位檔案

- Python一樣是使用`open()`函數開啟二進位檔案，只是開啟模式字串不同，如下所示：

```
fp = open("note.dat", "wb")
```

- 上述函數開啟檔案`note.dat`，第2個參數的模式字串多了字元"`b`"，表示開啟寫入的二進位檔案（讀取是"`rb`"）。關閉二進位檔案一樣是使用`close()`方法，如下所示：

```
fp.close()
```

將資料寫入二進位檔案

- 我們是呼叫pickle模組的dump()方法將資料寫入二進位檔案，如下所示：

```
pickle.dump(11, fp)
```

```
pickle.dump("peter", fp)
```

```
pickle.dump([1, 2, 3, 4], fp)
```

- 上述程式碼依序寫入整數、字串和一個清單。請注意！寫入順序很重要，因為後面需要以相同順序再將資料讀取出來。

從二進位檔案讀取資料

- Python是使用pickle模組的load()方法從二進位檔案讀取資料，首先開啟讀取的二進位檔案，如下所示：

```
fp = open("note.dat", "rb")
```

- 上述程式碼使用open()函數開啟檔案，第2個參數"rb"是開啟讀取的二進位檔案，然後呼叫pickle模組的load()方法來讀取資料，如下所示：

```
i = pickle.load(fp)
```

```
print("讀取整數 = ", str(i))
```

```
str1 = pickle.load(fp)
```

```
print("讀取姓名 = ", str1)
```

```
list1 = pickle.load(fp)
```

```
print("讀取清單 = ", str(list1))
```

例外處理

- Python例外處理程式敘述主要分為**try**和**except**二個程式區塊，其基本語法，如下所示：

try:

產生例外的程式碼

except <Exception Type>:

例外處理

- 上述語法的程式區塊說明，如下所示：
 - **try**程式區塊：在**try**程式區塊的程式碼是用來檢查是否產生例外，當例外產生時，就丟出指定例外類型（**Exception Type**）的物件。
 - **except**程式區塊：當**try**程式區塊的程式碼丟出例外，我們需要準備一到多個**except**程式區塊來處理不同類型的例外。

同時處理多種例外

- Python例外處理程式敘述有1個try程式區塊和3個except程式區塊，如下所示：

try:

```
n1, n2 = eval(input("輸入2個整數(n1,n2) => "))
```

```
r = n1 / n2
```

```
print("變數r的值 = " + str(r))
```

except ZeroDivisionError:

```
print("錯誤: 除以0的錯誤!")
```

except SyntaxError:

```
print("錯誤: 輸入數字需以逗號分隔!")
```

except:

```
print("錯誤: 輸入錯誤!")
```

else和finally程式區塊

- Python的try/except例外處理程式敘述還可以加上else和finally兩個選項的程式區塊，其語法如下所示：

try:

產生例外的程式碼

except <Exception Type>:

例外處理

else:

如果沒有例外，就會執行

finally:

不論是否有產生例外，都會執行

使用raise關鍵字丟出例外

- 在Python程式可以使用raise關鍵字自行丟出例外。例如：丟出ValueError例外物件，如下所示：

raise ValueError("年齡一定是正值")

- 上述程式碼建立ValueError物件，建構子參數是例外說明字串。

os模組

- Python的os模組提供目錄處理的相關方法，可以取得目前的工作目錄，建立和刪除目錄，更名或刪除檔案。相關方法說明如下表所示：

方法	說明
getcwd()	傳回目前的工作目錄
listdir(path)	傳回參數path路徑下的檔案和目錄清單
makedirs(path)	建立參數路徑的目錄
rmdir(path)	刪除參數路徑的目錄
remove(path)	刪除參數路徑的檔案，刪除目錄會產生OSError錯誤
rename(old, new)	更名參數old的檔案或目錄成為新名稱new

os.path模組-處理路徑字串的方法

- **os.path**模組提供方法取得指定檔案的完整路徑，和路徑字串處理的相關方法，可以方便我們取得路徑字串中的檔名和路徑，或是合併建立存取檔案的路徑字串。相關方法說明如下表所示：

方法	說明
<code>realpath(fname)</code>	傳回參數檔名的完整路徑字串
<code>split(fname)</code>	傳回參數fname分割成的路徑和檔案字串的元組
<code>splittext(fname)</code>	傳回參數fname分割成的路徑（含檔名）和副檔名字串的元組
<code>dirname(fname)</code>	傳回參數fname的路徑字串
<code>basename(fname)</code>	傳回參數fname的檔名字串
<code>join(path, fname)</code>	傳回參數path路徑合併fname檔名的完整檔案路徑字串

os.path模組-檢查檔案是否存在

- **os.path**模組提供方法可以檢查檔案是否存在，路徑字串是檔案，或目錄。相關方法說明如下表所示：

方法	說明
<code>exists(fname)</code>	檢查參數fname的檔案是否存在，存在傳回True；否則為False
<code>isdir(fname)</code>	檢查參數fname是否是目錄，是傳回True；否則為False
<code>isfile(fname)</code>	檢查參數fname是否是檔案，是傳回True；否則為False

建立多執行緒的Python程式

- Python可以使用threading模組建立多執行緒的Python程式，在此模組提供Thread類別來建立執行緒，我們可以指定此執行緒呼叫的函數來建立平行執行的多個執行緒。
- 在Python程式首先需要匯入相關模組，如下所示：

import threading, time, random

- 上述程式碼使用「,」逗號同時匯入多個模組，time和random模組是用來建立隨機的暫停時間，threading是建立執行緒。

建立多執行緒的Python程式

- 我們準備使用2個執行緒同時執行display()函數，如下所示：

```
def display(name, num):  
    i = 1  
    while True:  
        time.sleep(random.randint(1, 3))  
        print(name + str(num) + " =", i)  
        i += 1
```

- 上述函數有2個參數，第1個是名稱，第2個是編號，while無窮迴圈是使用亂數決定暫停時間，然後顯示遞增的計數值。

建立執行緒物件

- 現在，我們可以使用**Thread**類別的建構子來建立執行緒物件**thread1**，如下所示：

```
thread1 = threading.Thread(target=display,  
                             args=("執行緒",1))
```

```
thread1.start()
```

- 上述建構子的關鍵字引數**target**是執行的函數，**args**是傳入此函數的2個引數，第1個是名稱，第2個是執行緒編號，在建立**Thread**物件後，就可以呼叫**start()**方法啟動執行緒。

生成器(generator) & yield

- 透過**yield**關鍵字可以定義一個生成器函數，可一次回傳一個迭代器(iterator)。

```
def infinite_sequence():
```

```
    num = 0
```

```
    while True:
```

```
        yield num
```

```
        num += 1
```

```
iterator = infinite_sequence()
```

```
print(next(iterator))
```

```
print(next(iterator))
```

```
print(next(iterator))
```

```
for i in infinite_sequence():
```

```
    print(i)
```

```
    if i >= 4:
```

```
        break
```

Call by address

- Python函數參數的傳遞是透過傳物件的位址，要小心內容更改產生的參考錯誤。

```
def mul(M, n, x):  
    for i in range(n):  
        for j in range(n):  
            M[i][j] *= x  
    return M
```

```
M = [[1,2],[3,4]]  
newM = mul(M,2,3)  
print(M)  
print(newM)
```

巢狀函數(nested function)

- 函數中定義另一個函數。

```
def make_pow(n):  
    def pow(x):  
        return x ** n  
    return pow
```

```
pow3 = make_pow(3) # Generate function  
print(pow3(2))
```